Data exfiltration methods, test bed and prevention

BY

PROJECT UNIT: PJE40

Contents

Figures

Tables

Abstract

Data exfiltration by bad actors is of increasing concern throughout the world. A test bed for scrutinising potential new exfiltration methods should prove a useful tool for both businesses trying to protect data and IT professionals seeking to demonstrate the need for good cybersecurity practice.

A data exfiltration test bed programme was developed to carry out a variety of both newly designed and previously described, but newly streamlined, methods, including image and video steganography, utilising 404 errors on web logs, time to live manipulation, source MAC address manipulation, IP options field manipulation, hiding data in email headers and timing control.

The test bed was used not only to demonstrate how these methods operate, but also to investigate ways of detecting their use. The default implementation of Snort rules as an IDS tool was not sufficient to detect the exfiltrations. Customised Snort rules, however, were shown to be effective in detection of the exfiltration methods studied.

Some problems were encountered in the development phase, particularly with the video steganography and source MAC address methods. Solutions are proposed to take this work further.

Given the UK government's recent warning (UK Government's daily Covid-19 briefing, 5pm, 5 May 2020) to the public over a rise in coronavirus-related cyberattacks happening across the world, development of this test bed and the suite of detection rules will be a timely addition to the cybersecurity toolkit.

Word Count: 11675

Chapter 1 Introduction

In this project the topic of data exfiltration will be investigated. Exfiltration can be generally be thought of as unauthorised copying, transfer or retrieval of data from a computer or server, normally with malicious intentions by the perpetrator. Using data exfiltration, bad actors can steal data from computers or servers without the owners of these devices noticing the thefts. The harm that can be done to business as a result of exfiltration can be both, financial – from fines by regulators and compensation to owners for the data loss – and, a subsequent loss in public confidence in the business.

This project will fully investigate methods of data exfiltration that have been proposed and discussed in the literature, seeking to optimise them in order to reduce the time taken for the data to be exfiltrated. It will also seek to create new methods of data exfiltration. Then, both groups of exfiltration methods will be incorporated into a program that will become a test bed for owners of any network that stores sensitive data.

To bring the work further into a real-world context, an investigation will also be carried out to propose countermeasures that can be deployed against the exfiltration methods found to be successful.

The resulting program from this project will have two main uses:

- to routinely test the configuration of an existing network security set-up for resistance to exfiltration of data
- to demonstrate to business leaders what attacks are possible on a network and bring the importance of computer security to the fore of any business's IT operations.

1.1 Aims

The aim of this project Is to develop and implement methods for exfiltrating data from a monitored network, through the creation and discovery of new methods or different implementation of currently know methods. Creating a variety of exfiltration methods and a program to test to see if they work, will allow better detection and prevention of exfiltration by businesses.

1.2 Objectives

The objectives of this project are as follows, numbered for later reference in this document, not to define their relative importance:

1. Design and implement new methods of data exfiltration.
2. Implement existing methods of data exfiltration and attempting to streamline the processes.
3. Create a program to exfiltrate data using the methods.
4. Attempt to develop detection countermeasures for any implemented methods.

Chapter 2 Literature review

Data security has come to the fore in recent years, as more businesses use more complex IT systems in their day-to-day operations.

According to a paper released by the UK's Department for Digital, Culture, Media & Sport, in 2019: "32% of business and 22% of charities identified cybersecurity breaches or attacks in the past 12 months" (Department for Digital, Culture, Media & Sport, 2019). The report also identifies that these cyberattacks have had negative effects on businesses. It states that, in 2019 the average mean cost for UK businesses was £4,180; increasing to £9,270 for medium-sized firms and £22,700 for large firms. This highlights the importance to UK businesses of cybersecurity. If the detection and prevention of cyberattacks or breaches is not taken seriously there could be large financial repercussions. The report concludes that the introduction of the General Data Protection Regulation (GDPR) has, unintentionally, put the focus on the security of personal data. Following GDPR will, however, only take the business to a certain point. To be secure, from an IT point of view, businesses will need to seek further information and guidance on cybersecurity.

As highlighted by Faheem Ullah *et al,* data exfiltration is the third stage of a cyberattack (Faheem Ullah, 2018). The first stage is researching the target to identify weaknesses. The second stage is the use of possible attack vectors. Using the information gathered from the research, an attack will be launched at the identified weaknesses; it could be either a network or physical-based attack. The paper then goes on to list Intrusion Detection System (IDS) and Security Information and Event Management (SIEM) tools as "fall[ing] short in dealing with data exfiltration due to unstructured and constantly evolving nature of data" (Faheem Ullah, 2018). This shows that even though these systems are in place and operational, the ever-changing nature of data and how it is stored means that without upkeep and correct use of the tools, data exfiltration is still a risk. It could also be seen as a reminder of the importance of research into new and unpublished methods of data exfiltration, as they may have been discovered by bad actors and not publlshed.

Figure 2.1 Noticeable data exfiltration from 2017 (Faheem Ullah, 2018)

The report (Faheem Ullah, 2018) includes figure 2.1, which gives details of the extent of data breaches for major US organisations in 2017 and the type of data being leaked. The leaked data mainly consists of: name, addresses; date of birth; phone number; and social security data, which is equivalent to National Insurance number in the UK. As these types of data appear in most of the breaches listed, using this data to test the data exfiltration methods would be a good course of action to demonstrate the real-world application of the methods.

Various methods of data exfiltration have been detailed in the literature. Xin Liao *et al* proposes an interesting method for hiding images in other images by using the differences in colour values for each pixel and running the information through an algorithm (X. Liao, 2020). This method looks at all the values of the pixels and works out the inter-channel correlations of the original image, which is how each pixel's colour value relates the pixels adjacent to it. The method uses this information to hide another image inside the original. These original images were raw colour images, which are larger than traditional formats such as JPEG or PNG. This method works and can be used to hide images but there is no implementation method suggested to allow data to be hidden in the images. A novel would be to translate this into a two-step process where the least significant bit method is used to encode data into an intermediate image and then that image is encoded into a final image using the methods described. This may lead to the data being harder to detect. And if detected the second layer of steganography may not be unravelled if a different algorithm was used to encrypt the data.

In an article by Sudeepa *et al,* a method of video steganography that randomly selects frames to apply image steganography to is described as a way to hide data in a less detectable way (Sudeepa K.B, 2016). In this attempt the authors use an AVI (Audio Video Interleave) video format of uncompressed data to get around the issue of losing data through compression and encoding. The problem with this is that uncompressed video files can become exceptionally large. This would not be an issue if every single frame were being used to encode data but, as some frames are being skipped, they are using up space without having any data encoded into them. Using uncompressed video, however, increases the total number of frames that can have data encoded into them. This is an interesting technique, which could possibly be implemented on compressed video by identifying the I frame [define it here?] and using these as an available pool of frames to encoded data into using the random method described.

A report by Haddon *et al s*uggests a method of exfiltrating information out of a network by using the DNS quires (David A E Haddon, 2019).This uses the https to send a request to a DNS where the requested domain, in this case part of the payload to be exfiltrated, is encrypted in the packet. So even if the web gateway were to log the request to the DNS, the information hidden in the encrypted part of the request means the web gateway would store no useful information about the exfiltration. Haddon *et al* also came up with a speed at which the data would be exfiltrated using this method of *"750k/min or 45MB/hour"* (David A E Haddon, 2019). This is a slow method as it is restricted by the amount of data allowed inside the DNS request. But a trade-off is made between speed and likelihood of detection, as the slower the data is exfiltrated the harder it will become to identify. The increased time taken also means that there is more data for an analysis to go through to look for any instances of data exfiltration.

Alcaraz *et al* proposed three widely recognised methods of covert data exfiltration; storage-based, behaviour-based and timing-based (C. Alcaraz, 2019). Timing-based methods use the time between events; modifying them to hide the information sent rather than hiding it in the data being sent. This paper does not go into fine detail about how this would be implemented. Nor does it specify, what type of system could be used to receive the information in a more complex way than simply using high or low time latency as binary representation for data transfer.

In an extension to timing-based methods, Mordechai Guri *et al* described and implemented an interesting method of exfiltrating data from air-gapped systems, by using the sound of the fans in the computer to extract the data to a system up to seven metres away (Mordechai Guri, 2020). In this implementation Mordechai Guri *et al* used an Android app as the receiving part, as this could conceivably get close enough to an air-gapped system without suspicion. This demonstrates how the simple idea of timing control can be taken and applied to unexpected systems. This also highlights the important of making data exfiltration systems that look like they belong where they have been deployed, thus attempting to hide data in plain sight.

Chapter 3 Project management

The project timeline will run as follows, determined by the two main deadlines set by the university:

- Early stage development and testing of the validity of the exfiltration methods.
- Keeping notes and rough drafts on methodology and implementation during the early stages to document the thought processes behind the decisions made while implementing the methods.
- Deadline one 18 March 2020: A working and completed program is to be demonstrated at a showcase event. Here the program will be demonstrated to potential users.
- Following the demonstration event, feedback on any areas of the interface or program will be followed up, including any problems that arise with user input validation, and corrections to the interface that may have caused confusion to users. Any bugs that may arise can be fixed at this stage.
- In parallel, the final project report write up will take place
- Version control and backups for the project will be handled using a private GitHub repository so that the progression of the project is saved off site, ensuring it is possible to work on the project from any computer.
- Deadline two 1 May 2020: Report submission.

3.1 Developing methods

The flow chart in figure 3.1 shows the basic method for development and the iterative stages that it will employ.

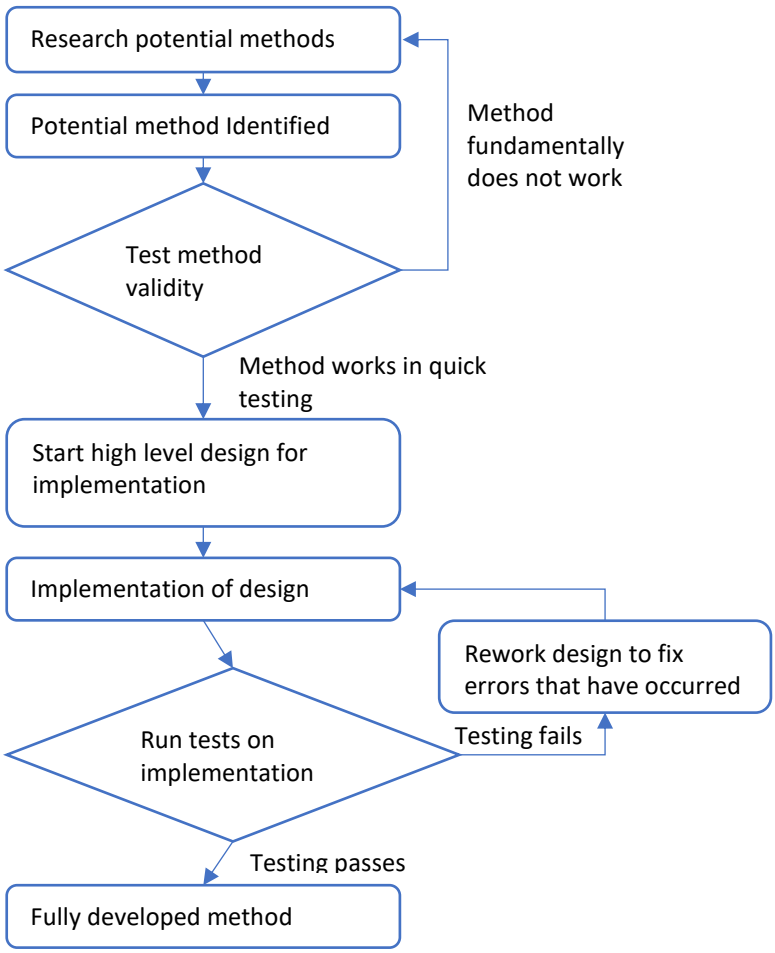

Figure 3.1 Method creation flowchart

3.2 Covid-19 outbreak in the UK

In March 2020, a worldwide pandemic of a novel coronavirus was recognised. The effect on society was to see lockdowns of citizens put in place across the world.

Because of the Covid-19 pandemic, the showcase event was cancelled at short notice. This decision was made in the week leading up to the event so there was no impact on the preceding timeline, as described above.

A significant loss to the project, however, could be that of any feedback generated from the session. As a result careful thought will be given to the level of expertise of likely users, to anticipate any such problems and solve them.

As well as the cancellation of the showcase event, and also as a result of the Covid-19 outbreak, the final deadline for the project was pushed back to 7 May 2020. The extra time could be used to compensate for any time lost due to illness, for example.

Chapter 4 Methodologies and project life cycles

4.1 Programming development models

The correct initial choice of an appropriate programming development model will greatly affect the outcome of any software development project. The main development models to be evaluated for the development of this project were Agile, Waterfall, Incremental and V models. They were chosen as commonly used development models, and each has some advantages and disadvantages.

The Agile model works in an iterative style where the planning, requirement analysis, design implementation and testing are carried out multiple times in that order. It places more values on the software working rather than having extensive documentation, which may be suitable in the fast-moving world of cybersecurity. It also provides the agility to respond to changes requested by a client very quickly, as the iterations include all the major steps for creating new programs. This is a useful property to enable quick changes to be made to a project. Once the analysis and high level design is implemented, however, there may be no need to change these in this particular project, so having to revaluate these parts of the project would add unnecessary delay to the process. For this reason, Agile may be a good option but it comes with some drawbacks.

Figure 4.1 Agile Development model Image from (Agile Model, 2020)

The Waterfall model, on the other hand, moves sequentially through the stages of software development. Once a stage has been completed then the development cannot move back to a previous stage. This can lead to a very rigid development environment that may not suit this topic. Here flexibility will be of use when, for example, it is found that methods seeming to work on a small scale do not work on a large scale. This model would not allow for any early-stage changes to improve the product or allow the developer to create new methods to replace ones deemed unsuitable.

The V model is an updated or continued Waterfall model that allows for review and rewriting of code and supporting documents. While this adds more flexibility to the structure, It still requires that all the designs and architecture are determined before the coding can begin. This added flexibility over the Waterfall model offers some advantages, in some types of project. However, it is likely that in the type of work anticipated for this project, ideas for new methods will arise while implementing methods already designed. This is not compatible with the V model as it would involve moving back from the implementation phase to the design phase, which goes against the model's flow.

The model chosen for this project needs to allow the developer to investigate multiple different methods in parallel, implemented over a long period of time. Using the Incremental model will allow development to focus on each of these methods at a time, meaning that all of the design, implementation and testing of each method is done and completed when the method is chosen. When one method is completed, the developer can move onto the next. This model, unlike the Waterfall model, also allows for building on and improving on previous attempts.

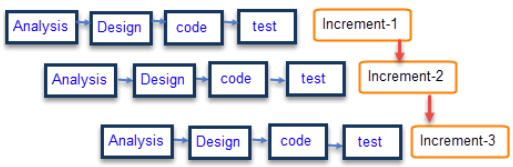

Figure 4.2 Incremental Model from (Incremental Model, 2020)

Thus, the incremental model best reflects the style of working anticipated as necessary on this project, so it is the model of choice.

4.2 Software architecture

Careful choice of system architecture is essential for the success of any programming project.

Based on previous experience in a cybersecurity work role, is it anticipated that most of the ways of exfiltrating data will involve two computers in direct communication, so it initially seemed plausible to see this project being done under the peer to peer (P2P) architecture. But this would restrict some possible methods for data exfiltration, such as any form of steganography, as this needs to be done locally and not across a network.

In pipeline architecture, the processes are seen as forming pipes that the data can travel down. This would allow some functions to be used multiple times over the different exfiltration methods. It is anticipated that many of the exfiltration methods will likely have a common core of logic that will need to be carried out in the same way before they diverge into what makes the method unique. This type of development lends itself best to pipeline architecture.

Chapter 5 Analysis and requirements

5.1 Analysis

The wide range of potential targets that can be exploited by cybercriminals during cybersecurity incidents means that it is not possible to precisely define a narrow range of file formats that any exfiltration program will encounter. In each case, the file format may be different. So, the exfiltration program should be able to read and exfiltrate files of any format without problems. If the program is designed with this in mind, then it will greatly increase its usefulness as a tool for emulating real exfiltration attempts.

Exfiltration attempts can be discovered if they cause any spike in network traffic. Allowing the user to specify and add a delay to the exfiltration program if the process uses network connections, will be an important feature. It will allow the users to better test the extent to which the exfiltration methods would be detected by the current system in place.

Each method used in the exfiltration program should be able to evade basic detection by an IDS or SIEM tool. As seen in the literature review above, the use of these systems by organisations in the real world is often not optimum. This suggests that it will be sufficient for the proposed exfiltration methods to aim to be able to evade detection by these systems set in their standard configuration.

In the final product, careful consideration must be given to how to convey the output of the exfiltration attempts to the user. The choice of interface is between a graphical user interface (GUI) or a command-line interface. In this type of system, the advantages of having the information represented graphically in a well-designed way will reduce any problems around users understanding how to operate the program. But if the program is designed to emulate the real world, then having a GUI on such a program would draw unwanted attention to it. This program is also being aimed at intermediate to advanced users, who should have some surface-level knowledge and understanding of the underlying systems. A further disadvantage could be the difficulty of creating a GUI, which will depend on the programming language chosen for the project.

The exfiltration program will need to be able to read a file from disk in binary form so that the data can be manipulated in a known way, sent over a network to a receiving device, and the reverse manipulation applied to arrive back at the original binary form. The manipulation used will vary depending on the exfiltration method, but the data should always be modified in some way before transmission. This is to add difficulty for any analysis or similar investigation that may be done after the data has been exfiltrated form the network. There is also a need for the exfiltration program to be able to make network connections, to allow the sending and receiving of data over a network.

As well as reading the file contents from disk, it would reduce possible user errors occurring if the file name and extension was exfiltrated along with the content of the file. If the extension is not correctly entered by the user, then, when the file is being reassembled, it may incorrectly indicate that the data stream has been corrupted in transmission. Also, by having the file extension within the data stream it removes the need to gather the information from the users and can speed up the process of reassembling the data.

5.2 Requirements

The listed requirements are numbered for later reference in this document, not to define their relative importance.

1. The program should be able to read and exfiltrate any file format.
2. It should be an option to add a time delay, between the sending of data specified by the user, in instances where the program is to use a network connection.
3. Each exfiltration method used in the program should be able to evade basic detection by an IDS or SIEM tool.
4. The program will use a command line interface.
5. All information including file name and extension will be exfiltrated.

5.2.1 Functional requirements

- Read file from disk in binary form.
- Modify the data before sending.
- Make network connections to other devices
- Modify network traffic before sending
- Monitor network traffic

Chapter 6 Design

6.1 Data exfiltration methods

The different methods used in this project can be divided into three main groups:

- Content: where the content being sent over the network hides the information.
- Header: where the information is obfuscated in the header of the packets being sent.
- Meta: where the information is encoded in the details of when and how the packet was sent.

6.1.1 Content

Sent content provides a place to hide exfiltrated information. When it comes to detection these methods may be the hardest to identify as information can be encoded without the content looking malformed. Repetition of content may lead to detection, if there are no time brakes added into the sending of the outputs from these methods, as the methods can cause spikes in network traffic.

Http 404 error

This first content method uses the 404 errors that are stored in the logs of web servers. If the logging is set up correctly, it should be possible to encode the binary string from the file. First the file is read from the disk in binary form. Then each byte is converted into a two-digit hexadecimal number. Ten of these sequential hexadecimal numbers are then concatenated and formatted into a URL, which is then sent to the user-defined host address. Then the web logs will need to be retrieved from the server and put through the program again to generate the file output.

Image steganography

Image files provide more complex files in which to hide information. If this methodology is implemented properly the output image should be nearly indistinguishable from the original image. With this method, the ideal design would use the least significant bit of each colour value of each pixel to hide data in. This would be done by moving down the columns of pixels encoding three bits to every pixel, putting one bit in the least significant bit of the values in the RGB channels. When the image has been manipulated it will not be noticeably different to the human eye because only the least significant bit has been changed.

For the reasons given it would be ideal to implement a method that only changes the last bit of information. It would be simpler to just increase the colour value by one every time, however this could affect any number of bits depending on the initial value. This is shown in table 6.1 where if the value in the pixel is 119 and the program just adds one to change the least significant bit then it would modify the last four bits of information.

Decimal	Binary
118	01110110
119	01110111
120	01111000

Table 6.1 Changes in Binary values

A better way to modify the least significant bit would be to look to see whether the initial value is odd or even. If it is odd, then the least significant bit will be a one. If it is even, then it will be a zero. In the example provided 119 is an odd number so it should subtract one making the colour value 118, having only had to modify the least significant bit. From this logic, a set of statements could be set up to ensure that only the least significant bit was changed.

Once the image has been encoded with exfiltration data, a similar process must be run in reverse to get the information out of the image and write it back onto disk as a file.

Video steganography

Video files provide an alternative picture-based method that allows for more information to be hidden within a single file. Here the image steganography functionality can be applied to a selected frame in a video file, generating similar output for hiding the information. For this to work each I-frame – an intra-coded frame where the whole frame is encoded like a standard still picture – in the video would need to be identified. This is because not all frames in most encoded videos are full images, they are just predictions of

what the frame will contain called P & B frames. Information cannot be hidden in these frames, as it could be in an image, because these frames are instruction sets for the video decoder and not image-like.

The program will need to identify all I-frames in the video and then encode the data into them using the method described for image steganography, above. Implementing this idea of choosing certain frames to encode data into the method described by Sudeepa *et al* (Sudeepa K.B, 2016) would have the advantage of making detection of the modified data more difficult.

6.1.2 Header

Packet headers provide the opportunity to hide data, for example by changing the expected values to encoded bytes of data. In this way, it might be possible to hide data in more unexpected places, but the speed of data transmission would be sacrificed, as there is less space in the headers and therefore less space to hide data.

Time to live (TTL)

The Time to Live field can be used to hide information in a previously little thought of way. This can be done, by adding the value of a byte to the pre-set time to live, which will always be the same. This will allow the receiving component to subtract the fixed time to live value from the received value and then convert the result into a binary number for the byte. By reassembling the binary numbers gathered from the TTL fields into an array, a copy of the file will be produced, ready to write to disk.

It will be necessary to send the file name and extension along with the content of the file. This can be done, by asking the user to supply two separate addresses, to spoof the source address of the packets. Then they can be identified by the receiver and added to different arrays. The two arrays generated would be decoded to retrieve the respective information that they hold. This method of sending the file content and file name will also be used in other methods when there is a source address that can be manipulated.

IP options field

This method revolves around hiding obfuscated information within the options field of the IP protocols. Some of the available options have no restriction on what can be included in them, except for a maximum length. Such a custom option, with no maximum length and no formation restrictions, might be a suitable candidate for attempting to have data hidden within it. An advantage is that the speed of this exfiltration method should be faster than other header methods. It is not limited to any size restriction of the amount of data being sent in one packet, unlike in the case of TTL, which is limited to one byte of information per packet sent.

The program will consist of two main parts: the sending and receiving parts. The sending part should handle the deconstruction of the file and crafting the packets that contain the information. While crafting the packet, it would be an advantage to spoof the sender's IP address. This could be used to indicate which packets are file name and which are file content. Then the program should send the packets to the destination using any delay that the user has added through the initial inputs given. The receiving part will monitor for packets coming from the IP addresses supplied by the users. When all of the packets have been received the program will need to extract and reassemble the data.

Source MAC address

This method relies on the ability to change the source MAC address in the Scapy library. The fact that a standard MAC address is six bytes makes it a suitable choice for hiding data in. It is a field that is expected to contain some form of information, and in this instance it could be information that is being exfiltrated from the network. This strategy should reduce the number of packets to be sent compared with the majority of the methods studied in this project, which only send one byte per packet. The significant increase in speed and reduction the amount of traffic being sent across the network, would in turn reduce the chances of this method being detected.

This method would convert the files' binary array into MAC address formats and then attach each to a packet and send it to a receiving computer. Once received the MAC address should be stripped from the packet and decoded back to binary then reassembled into a binary array to be written as a file to disk.

Email headers

Custom fields that will not show up when only taking a quick look can be added to the headers of emails. Information could be stored there without being noticed, effectively hiding information in plain sight. Such a manipulation could, however, be spotted by a more discerning user. So, it would be advisable to encode the data in some way, to avoid the user realising its significance. This could be as simple as encoding it using a normal encryption algorithm such as AES. A second option is to convert it into what looks like another set of information such as MAC addresses, which would reveal incorrect information to any investigator.

6.1.3 Meta

In this category, the data is hidden in information that is not inside the packet at all. The main example is the use of timing control, where the time between packets being received indicates what has been encoded using a predefined schema.

Timing control

In this method, the information in the packet is irrelevant and the program will look at the time difference between when different packets are received. It will need to be kept in mind that the intervals will not be constant, as packets will likely be routed through different paths causing differences in the sending and receiving intervals. This will need to be tested to determine the best value for delay, so that speed can be maintained without introducing errors resulting from routing taking so long.

6.2 Programming languages

The choice of programming language for this project has come down to one of two – Python or C++ – as the developer has experience of using both of the languages. Both languages can communicate with the lower level parts of the networking layers, which is required to complete this project. Both have higher-level language utility that would be useful.

Python was eventually chosen because, within the skill pool of this project, the developer was more experienced in using the language. Also, there is a good variety of publicly available libraries that can be used to implement some of the functions required. The main example is Scapy, which will used to craft, send and receive packets. Python also has the ability to be able to modify many of the various options required for the implementation of the methods described above.

Chapter 7 Implementation

7.1 Data exfiltration implementation

In all of the proposed methods of data exfiltration there is a common theme of gathering the user inputs using a singularity function call, usually called “user_inputs()”. This function call is the clearest way that this can be done. This approach also allows this functionality to be compartmentalised away from the logic of the program, causing fewer bugs. It also means that during late-stage testing and debugging the variable can be hard coded into the code to speed up testing without having a major impact on the code.

For all of the methods, it is also necessary to check that there is no file with the same name as the output file from the program. The function that completes this is “does_file_already_exist(proposedFileName)”. In this function, the proposed file name is checked to see if it is already a file in the directory that the Python program is in. If a file with the same name does not exist then the proposed file name is returned. Otherwise a number in pair of brackets is added to the end of the file name before the file extension. If at the end of the file name a pair of brackets with a number is found then the number is read from that file, one is added to it, then this new value is put back into the function; making this a recursive function that will only ever return a unique file name.

7.1.1 Content

Http 404 error

The user is initially prompted to select between the 2 modes of this method, send and reassemble. When the user selects send the users is asked to give the required information, which are file to exfiltrated destination web address and possible delay. When the destination web address is collected a simple get request is sent to attain whether the server is reachable or not (to see more information on this process see 8.2 User input validation). Once these have been gathered, the file is then read from disk as a binary array and is then converted into a hexadecimal array where each eight-bit binary number has been converted into a two-digit hex number. Each two-digit number will occupy its own element in the array. Then the hex numbers are combined into strings of a length that is defined by the hardcoded value of “URLlength“. This is normally set to 500 and can be changed at the top of the source code. When a URL of the specified length is created it is appended to a new URLs output array. Once all of the file content has been converted into this URL format then a part number is appended onto the front of the URL separated from the concatenated hexadecimal values with a “/”. The file name goes through a similar process where it is converted into hexadecimal representation of ASCII and then in added to a URL with the part number of “n”. This URL then gets appended to the URL output array. Now the program moves on to sending the URLs using the user-defined delay required.

The testing done with this implementation used an Amazon web services (AWS) S3 bucket with public permissions to view the contents as a static web page. The S3 bucket also has server-access logging enabled, to log all connections and requests to the S3 bucket. This makes it act in a similar way to a web server and its logging conventions.

As this log information is coming from a S3 bucket, which is a distributed system, the logs come through in multiple files with a high likelihood that they will not be in the correct order. Because of this the program must also include an option to concatenate all files in a directory into a single file. This misordering also highlights the importance for the part number to be in the data stream, as without it the information would be reassembled in the wrong order.

Once the data has been sent through to the web server, then the log file can be retrieved from the web server and the program can reassemble the data. It will use the standard format of the data to locate the information inside the log file. In testing this method an S3 buckets, the logging stores more information than needed (this information being a unique identifier for the S3 bucket and its name. It will extract just the requested URL into a new array. Then the program will use the part number embedded into the URLs to

piece them back together in the correct order. The program then writes the bit array to disk, which will create an exact copy of the file that was read from the previous run of the program.

Image steganography

When this method is initiated the user will be asked for the image file to be used to hide the data. This can be any in major image file format. The output of the image with the data encoded into it will be as a PNG, as this is a lossless format and the all-important least significant bit will not be lost unlike with more common formats such as JPEG.

When all the information has been provided by the user, a check will be run comparing the data that the user wants to encode into the image and the dimensions of the image to see if all of the data can be contained in it. If it cannot be, the user will be asked whether they want to continue or abort the run of the program. Continuing will only encode the amount of information that the photo can hold by truncating the data when the space runs out.

In this setup of Image Steganography, the file is read from disk and given to the program as a byte array, which is then turned into a string of all the bytes concatenated together. Then the full file name, including the file extension, is converted to utf-8 binary. Then 32 bits of zeros are added, to separate the file name from the file content. The program looks for this when reassembling the file when decoding. This is then appended onto the front of the file content, and another 32 bits of zeros are added to the end of the file content, to signal the end of encoded data.

The image supplied by the user is opened using C2V Libraries, where the image is represented as a two-dimensional array. The x and y coordinates of the image are the two dimensions of the array, and in each location there is another array storing the RGB values of the specific pixel. So, when the data is being written to the array, it starts at the top left pixel, moves down the columns and then moves left.

When encoding the data, the program will look at the value to be encoded into the colour value. If the digit is already set to the correct value, then the program skips it. Otherwise, it changes that value so that only the final digit of the byte needs to be changed. Then it moves onto the next value to change and repeats the process.

Once all the data has been encoded into the array it is then output to a PNG with the file name given by the user. This exporting process is done with the highest level of compression available to PNG images. The only disadvantage here is that it can take longer to create the file as the compression algorithm is more complex. This will also affect the read time of the file, as the uncompressing is also more complex than for a standard PNG image, but this does not affect this project.

Figure 7.1 Input image

Figure 7.2 Output Image

As seen in Figure 7.1 and Figure 7.2, the input and output images are indistinguishable from each other by the human eye. The output image has a CSV file with 50,000 rows of data, 5.44MB in size, encoded in it. In

this example, the image started as a PNG image of 26.1MB and the output was only 25.6 MB. This could be because the original image was only mildly compressed, but it does show it is possible for the output image to be smaller than the input image.

When decoding an image, the user enters the location of an image with something encoded into it, then the program opens the image in the same way as when encoding data. But it reads the last bit of every colour in every pixel. When it has read into the file to detect two sets of 32 zeros, the program stops reading the image and closes it. It then takes the string of data generated between the start of the file and the two sets of zeros. It decodes these two pieces of information separately; starting with the filename, which is returned to a string, and then the content of the file, which is returned to a binary array ready to be written to disk. Then, with that information, it creates an identical copy of the file to disk with the same file name and extension

Video steganography

Implementing this method should have been a simple matter. Having worked out all the difficulties with the image steganography described above, it was anticipated that they could be reused, however, an unexpected difficulty was encountered. There is no pre-existing method for identifying I-frames in a video file in Python. It is not possible to predict when I-frames will occur as their position varies depending on the compression and encoding methods used. Therefore it turned out not to be feasible, within the scope of this project, to attempt to create functions that could identify I-frames. The basis for the idea could still be used as an exfiltration method, but it would take considerable time to develop the I-frame detection system.

7.1.2 Header

IP options field

The biggest option field that is not a custom field has a maximum length of 20 bytes, so in order to transmit more information for each packet sent an initial attempt was made to use a custom-length option field. But the prototyping showed that the custom-length options could become malformed rather frequently and so the decision was taken to use a pre-existing, unrestricted options field. The specific option used is the “RFC3692-style Experiment”, which has a maximum length of 20 bytes.

When the user selects the sending mode of the program, this collects the information given by the user and uses it to read the file for exfiltration from disk. It is converted from a binary array to a hex array and the file name is encoded into ASCII and then into hexadecimal. This file content hex array is joined into 20 bytes and then a packet is crafted with the information given by the user and the packet and added to a packet array. Once all of the packets have been created, the array of packets is iterated through sending all of the packet in the array using the delay specified by the user, if applicable.

When the listening part of the program is set up, the user gives the program the two spoofed addresses for the file name and file content. After this information has been given, the program will listen for any packets from the two addresses. When the user terminates the listening part, the program takes all of the packets collected and puts them into two lists depending on the source address. Once this is done, both of the lists are decoded to reveal the relevant information, and then the file is written to disk using the file name and content.

Time to live (TTL)

This method will use the TTL field to encode information. This is done by having a base value of 64 and then adding the decimal value of the byte to the base value. With this the TTL can vary from 64 up to 319. The program reads the file given by the users as a binary array of eight bits. It takes these binary numbers and turns them into a decimal array with the same order. Then it crafts packets with the decimal value added to each packet’s TTL. These are spoofed for the file content source address that the user provides.

The file name and extension are then encoded into UTF-8 and converted to a decimal array. This is then added to the default TTL and crafted into another set of packets. This time the source address is spoofed as the second address given by the user.

Once all the packets have been created, they are sent through to the destination address the user has provided. Then the program sends out the packets. If requested by the user, it uses the slowing functions to add random deviation to the times that the packets are sent, ensuring so that there is no noticeable jump in the network traffic of the sending machine, which may otherwise draw unwanted attention.

The receiving part of the program must be given the two spoofed source addresses: one for the file content, and the other for the file name. Once these two parameters are entered then the program starts listening for traffic from these addresses. Anything received from them is saved. Once the packets have been sent from the sender, the listening part is stopped by the user. The program then reassembles the data from all the packets received, starting with the file name. It looks at all the TTL of the packets and then spits them into two arrays by source address. Then for all packets in the arrays, the predefined value of the TTL – in this case 64 – is subtracted from the received TTL value leaving the decimal representation of a byte from the file or file name depending on which array. These are then converted to binary representation and the file name array is decoded into plane text. With the file name and file contents the file is then written to the disk.

Source MAC address

After testing this method in an network between two computers and implements a simple system with no users inputs, it became apparent that when this was tested over the internet to devices outside of the local network the source MAC address field would be overwritten by the last router to route the packet. Meaning that the information would be lost. When this was discovered the development of this method was stopped.

Email headers

For this exfiltration method, the user needs to have access to a Gmail account that is set up so that programs can access and use it through Google account security. Once this has been set up correctly the program will work.

The method uses the ability to hide data in custom headings of emails that do not show up when the email is viewed. Initially the users are asked for the login details of a Gmail address. Once logged in, the user is asked for a file location, receiver's email address and subject line. The subject line must be unique as this is used to retrieve the emails by the reassemble part of the program.

The program reads the file as a binary array and then converts it into two digits of hexadecimal and then puts these hexadecimal values into an array. The array of hex numbers is converted into eight number chunks. This makes them look like MAC addresses, hopefully causing any person that finds them to start looking in the wrong places, believing them to be MAC addresses and not a data stream. Once these have been created, up to 40 are added into the headers of an email, which is then sent with the subject line given by the user.

The reassembling part of the program asks the users to input the email login of the receiving email account and the subject of the emails sent with the hidden data. The program contacts the mail server and retrieves all the emails that match the given information. Then the information is extracted from the emails and reassembled. If there is more than one email, the program uses the time stamps on the email to determine the correct order to process them in.

Once all the information has been extracted from the emails it is reassembled into the original binary array and then it is then written to the disk as an exact copy of the original file.

As entering email and password information can be time consuming, a config file system using .ini files has been implemented into the program. The .ini file format was chosen because the base install of Python 3 has the ability to read and interpret this file type.

In this implementation, the receiving and decoding part of the program only works with Gmail accounts. In future, this could be changed to make the method work with other account providers, or a self-hosted email server, simply by changing the specified SMTP and IMAP setting to match the provider.

7.1.3 Meta

Timing control

This method revolves around looking at the time delays between when packets have been sent. When looking at all the packets sent there should be two distinct groups of results. The cluster that shows the shortest time between sendings indicates that the data should be read as a zero and then the longer gaps should be read as a one.

When this program is run the user can select one of two modes: sending; or receiving, which does the receiving, decoding, and reassembling of the data.

The sending mode askes the user for the destination address of the listening computer, and the location of the file that will be sent. The program then reads the binary array of the file on disk and converts that into a string containing all of the binary numbers. The program iterates over the string and for each bit in the string it reads if it's a one or a zero. If it's a one, it adds an artificial delay to sending the packet. The packets are sent using TCP/IP protocols.

The file name and extension is sent as the message of the first packet sent to the listener. This is looked for and used by the receiving part to determine the bit string. When the last bit is read from the string, a halt packet is sent as the last communication. This signals to the listening part that all parts of the data have been sent and analysis of the packets can begin.

7.2 Data generation

When testing these methods, it would have been possible to use random data but, as identified in the literature review, to get a more realistic data set for testing the method it would be better to attempt to remove some data from a system. This led to a search for a corpus of some kind that could provide information of a type that would be considered sensitive but with no ethical implications attached to its use, as it was not real.

After conducting this search, no corpus of a good enough standard was found, which led to the development of a function that could generate data to stand in for personal data. The program produces a file of any length requested by the user that contains the following items: names, addresses; date of birth; phone number; national Insurance number; and credit card information including card number, security number and expiry date.

Chapter 8 Testing

8.1 Logic tests and debugging

For all the exfiltration methods developed, the main debugging was carried out with a small 1KB CSV file generated by the fake data generator. This was used to test that the logic of the code worked successfully. This was chosen as the main testing during the creating and initial debugging phases, as it did not take a long time to complete the transfer between the two devices. This increased the speed at which the debugging could take place, as less time was spent waiting for the program to iterate through large amounts of data. Any logic errors could be spotted on the smaller files in a quicker time. To test if the input and output files were identical, the files where hashed and the hashes compared to check if they matched. If they did then the program was considered to have passed the test. These small-scale tests, carried out during the main development phase, were done between two computers on the same network.

Once the code was fully implemented and running without bugs for the small file, a larger CSV file was used to test whether increasing the size of the input file would break the code in unexpected ways. If the increased file size had no adverse effect, then different file formats were used, normally consisting of a PNG image and a zip file.

Using a zip file would confer the advantage of reducing the sizes of the file and in turn reducing the time taken by the method to transmit or encoded the data. Also, if there was a problem with sending particular files types then the file causing the issues could be added to a zip archive and then sent through the methods.

The PNG image file was selected for use in testing and debugging as the image steganography method could be applied initially and then the output of the steganography could be used as the input to any other method. The steganography method always outputs a PNG image and doing this may aid in obfuscation of the data. If all these files where correctly transferred, then it was inferred that any other file format would also not cause bugs in the code and the method's code could be considered complete and ready for use.

8.2 User input validation

As the program calls for user inputs, it is critical to test the error handling for each of the possible inputs. The inputs can all be simplified to: file location; directory; users added delay; IP address; and domain name. As these are the common inputs, the test and validations will be the same no matter where the request for the users input appears in the program it will always use the same function to handle them.

The program uses the function ValidFileName(filename) to validate a file location given by the user. The function looks to see if the given file name is empty. The path function in Python will error and brake if the input is empty, and the function will return false. If the file name is not blank, then the program uses the path.is_file() function to return true if there is a file with that name or false if no file is found.

The function to validate directory, is similar to the validation of file locations but uses the path.isdir() function instead of the path.is_file() to give the same output.

To validate user's added delay, the input given by the user is tested to obtain whether it is a digit or not. if it is not a digit, then the user is asked to enter a new input that is a digit, and this is repeated until a digit is entered. After a digit is entered, if that digit is a negative number the user is informed that the delay will be set to zero as a negative delay is impossible.

To validate IP address, the user's input is split into an array using "." A valid IP address should return an array with length of four. If the input returns the correct length array, then each digit in the array is tested for whether it's a digit or not, and finally the digit is tested to be between one and 255. If all of these tests are completed successfully, then the IP address is deemed valid. If at any point the tests fail then the input is deemed not to be valid as an IP address. If this happens the users is asked for a new input and the process starts again.

To validate domain name, the program takes the user's input, adds the required "http://" to the domain if it is missing, then attempts to make a simple get request. It uses the response from the request to decide whether the domain is reachable. If a 200 code is received the domain is considered valid. For all other outcomes, the status code is given to the user within the error message generated by the program.

8.3 Snort testing

For this project, Snort was chosen as the network intrusion detection system that the methods would be tested against. The developer had experience of using, creating Snort rules and setting up an instance to monitor a network, giving advantages over other available alternatives. That Snort is open source also contributed to making it the most obvious choice.

The most stable release of Snort – 2.9.16 – was used. It was set up in a virtual machine and the program was run inside this machine, and attempted to connect to another device on the network running the receiving parts of the program.

Chapter 9 Evaluation

Requirement one, that the program should be able to read and exfiltrate any file format, has been satisfied in all methods included in the test bed program. As described in the testing section, above, a carefully chosen range of file formats was used to prove that all possible file formats will be accepted by the program. This is aided by functional requirement one, which states that the file is read from disk and returned as a binary array. This allows any file format to be exfiltrated, as the program does not attempt to open it.

Requirement two, to provide an option to add a time delay, between the sending of data specified by the user, has been included in all methods except for image steganography. This exfiltration method does not use network traffic to encode data, so there is no need to add the delay as the requirement specifies that it will only apply to methods that use network connections.

The program uses a command line interface to obtain and display the user-supplied information, as specified in requirement four. Requirement five has been satisfied as each method has been successfully designed and implemented so that the file name and extension are in the data stream that is being transferred.

Requirement three, reflecting the aim of bad actors to evade detection, led to an important consideration of ways to improve the detection of exfiltration methods, as discussed below (section 9.1) These important concepts can be applied by businesses.

9.1 Snort rules

These sets of tests check whether requirement three, to evade detection, has been fulfilled. For the test, the Snort community and registered rule sets were used to test against. These are the two freely available rule sets that Snort provides. It does provide a third set of rules called subscriber rules, but they require a subscription to snort which was not in the available resource pool of this project.

In Snort, a suspicious packet can be in one of two states: either blocked or alerted. The blocked state indicates that the rules detected that Snort should stop all traffic. The alerted status is used to indicate that the packet may be suspicious and should be investigated further.

When ranking a method, it would be considered to have failed if it were blocked by the community rules. It would be considered a partial success if the method were only alerted to, as well as varying in success depending on how many of the packets are alerted.

Table 9.1 Method detection testing

Method	**Passed community rules?**
Http 404 error	Yes, without being blocked or alerting
Image Steganography	Not applicable as does not use network traffic
Time to Live (TTL)	Yes, without being blocked or alerting
IP Options Field	Yes, without being blocked or alerting, although they do appear in the “other” section when looking at the breakdown by protocol even though they are sent using IP/TCP
Email Headers	Yes, without being blocked or alerting
Timing Control	Yes, without being blocked or alerting

Table 9.1 shows the results of the Snort testing. Requirement three to evade detecting has been met, as none of the methods were blocked or alerted by the IDS.

As it was found that all of the exfiltration methods successfully evaded the current rule set of Snort, a new set of rules had to be developed to identify these methods of data exfiltration to meet the aim of proposing to develop detection countermeasures for any implemented methods.

To combat these methods evading the NIDS rule sets, the rules laid out in Table 9.2 were put together to combat the exfiltration of data by identifying the packets sending the information. These new rules were then tested. All methods except timing control were found to be blocked by the newly created Snort rules. The note on timing control in table 9.2 explains the difficulty faced.

Table 9.2 includes notes on relevant information about other countermeasures that may need to be implemented in conjunction with Snort. These would increase the accuracy of alerts by reducing the false positive rates that the new rules will likely cause when applied in an enterprise setting with large and varied network traffic. Applying a SIEM tool, as described, would collect the snort logs and preform automated analyses on the logs.

Table 9.2 Proposed detection methods

Method	Snort rule	Notes
Http 404 error	alert tcp any -> any any (content:" 404 Not Found"; width: 30; sid:3000001)	There is probably a need to add a SIEM tool to review the information generated from this, as this will flag on all 404 errors. The tool could look for multiple 404 errors from the same source address, to the same destination, over a period of time.
Time to Live (TTL)	alert tcp any -> any any (ttl: ![0, 64, 128, 256] ; sid:3000002)	By looking for non-standard TTL times, it may be possible to identify packets with information being sent through them. For better identification of these instances, an SIEM tool could be used to evaluate the number and consistency of the packets coming from each device on the network. That information can be used to highlight any abnormally large number of packets coming from a single device, as this would indicate this type of data exfiltration.
IP Options Field	alert tcp any -> any any (ipopts: any; sid:3000003)	As this is an experimental options field, there should be a small number of false positives using this rule.
Email Headers	alert tcp any -> any 25 (msg:"attempted use of port 25 for SMTP"; sid:3000004) alert tcp any -> any 456 (msg:"attempted use of port 456 for SMTP"; sid:3000005) alert tcp any -> any 587 (msg:"attempted use of port 587 for SMTP"; sid:3000006)	A simple web gateway block on all SMTP and IMTP except thoughts used by the organisation would stop any unseen exfiltration happening as organisations can log emails sent and received by internal email addresses
Timing Control		To identify this, Snort would not be able to detect this as Snort only looks at packets in isolation and not as part of a whole network traffic. So, this detection would have to be boosted up to the SIEM tools and would probably have to involve looking for something similar to beaconing. The basic prevention for this is to look for single device on the network that are attempting to call out the same destination over a long period of time, i.e. one day. Then if there are enough callouts within the given time frame the SIEM tool can flag them as suspicious.

Chapter 10 Conclusion

Some theoretical assumptions were made when creating the designs for the exfiltration methods worked on in this project, and an important part of the project has been to refine the theories to provide working methods that will be of use in the prevention of data exfiltration.

The exfiltration methods developed using Image steganography, TTL, email headers and timing controls, were implemented exactly as described in their original designs, without any need for change, as the assumptions made proved to be correct in the real-world application of these methods.

However, some of the proposed exfiltration methods investigated ultimately failed to meet the project requirements, as the assumptions made were not correct or did not hold up in real-world scenarios. This highlights the importance of the practical nature of the testing carried out. They included the video steganography, MAC address and HTTP 404 error methods.

In video steganography, which failed at an early stage, the inability to identify I frames caused the method to become a burden on the resources of the project. The MAC address approach passed the initial small-scale test done within the same network, but went on to fail in the larger tests across the internet. The problem identified was that within the header of packets the source MAC address is replaced by the last device to route the packet, rather than the original source.

The IP options field method worked in the real-world scenarios, following some small changes that needed to be made to the theoretical design. When originally proposed this method used the “custom” options field, which has no restriction on the size and format of data inserted into it. But it was discovered that this field could become mangled and occasionally truncated when transmitted over a network. This led to a change to using the experimental field, which has a restriction on length but is not restricted in format of data, unlike the other available options.

The last small change from the proposed methods identified during the early testing phase, involved the 404 error method. Here the addition of a part number was needed as some web logging systems, such as the one used for testing, do not have the logs appearing in chronological order. Without a solution, the output file would have been reassembled incorrectly.

Most useful in real life, the objective to develop detection counter measures has been successfully achieved, as shown by the testing work carried out during the evaluation phase of the project. Initial tests of the methods against the default rule set of Snort were carried out and the rules failed to detect the exfiltration. However, following on from this work, the deep understanding of how the exfiltration methods work, gained from the development stage of the project, was instrumental in constructing a new set of Snort rules that better detect the exfiltration of the data using the specified methods. In practice, in conjunction with a SIEM tool these methods have become detectable.

10.1 Looking forward

With extra time and resources, the problems encountered with the Video steganography techniques – not being able to identify the I frames, as discussed in chapter seven – could be solved. This would lead to successful implementation of this technique of data exfiltration, as the rest of the encoding method has been developed and implemented in the image steganography method. All that would be needed is a system to chunk the data into parts and then hide these parts in the selected frames of the video.

For the 404 errors method, development of a different method to identify the correct order for the checks of information stored in the web logs is needed. This could be done by looking at the time stamps on the events and then ordering them into the correct order using that information rather than relying on the part number that had to be added to the URLs.

It would also be interesting to study the meta-type methods further. Combining all of the header methods into one single run on packets could bring advantages, as doing this with the IP options and TTL methods was found to increase the speed by reducing the number of the packets to be sent for the file size.

Hopefully, the findings in the project and the program created will be used by IT professionals to bring the issue of data exfiltration to the forefront of cybersecurity planning and actions within businesses.

Chapter 11 References

Agile Model. (2020, April 10). Retrieved from Javatpoint: https://static.javatpoint.com/tutorial/software-engineering/images/software-engineering-agile-model.png

C. Alcaraz, G. B. (2019). Covert Channels-Based Stealth Attacks in Industry 4.0. *IEEE Systems Journal, vol. 13*, Pages 3980-3988.

David A E Haddon, H. A. (2019). Investigating Data Exfiltration in DNS Over HTTPS Queries. *IEEE 12th International Conference on Global Security, Safety and Sustainability (ICGS3).* London, UK: IEEE.

Department for Digital, Culture, Media & Sport. (2019). *Cyber Security Breaches Survey 2019.* Department for Digital, Culture, Media & Sport.

Faheem Ullah, M. E. (2018). Data exfiltration: A review of external attack vectors and countermeasures. *Journal of Network and Computer Applications 101*, 18-51.

Incremental Model. (2020, April 10). Retrieved from Guru 99: https://www.guru99.com/images/6-2015/052615_1049_WhatisIncre2.png

Mordechai Guri, Y. S. (2020). Fansmitter: Acoustic data exfiltration from air-Gapped computers via fans noise. *Computers & Security Volume 91*.

Sudeepa K.B, R. K. (2016). A New Approach for Video Steganography Based on Randomization and Parallelization. *Procedia Computer Science, Volume 78* , Pages 483-490.

X. Liao, Y. Y. (2020). A New Payload Partition Strategy in Color Image Steganography. *IEEE Transactions on Circuits and Systems for Video Technology, vol. 30*, Pages 685-696.

Appendix A

School of Computing final year project

PJE40

Project Initiation Document

Data exfiltration methods and testbed.

Project Initiation Document

1. Basic details (mandatory)

Student name:	
Draft project title:	Data exfiltration methods and testbed
Course: Project	Computer Science
supervisor:	

2. Degree suitability (mandatory)

This project is suitable for a computer science degree as computer science is a broad degree topic allowing for a large margin to gain interests in particular area, This leads to the possibility to being able to engage in this project.

The projects main knowledge areas are; networks & cyber security. These are both covered by computer science.

3. Outline of the project environment and problem to be solved

The environment for this project is penetration testing of network defence. The intended audience for this is any persons who are responsible for keeping data on a network safe and they will be using this tool to test that,

4. Project aim and objectives

The overall aim of this project is to develop software that tests multiple methods, some of which I shall have come up with of data exfiltration from a network, then have the software decode any encoding methods.

5. Project deliverables

The Deliverable for this project are;

The Data exfiltration and reassemble program A set of test data.

How to add more Data exfiltration methods to the system.

Project report.

Explanation of how the data exfiltration methods work as well as instructions of how to reassemble the data.

Details descriptions of the methodology used to exfiltrated data using the methods I have created.

6. Project approach

The first research to undertake is what types of solutions already exists and when method of data exfiltration they use. Work out what algorithms they are using to implement this and then implement them into my system.

Then to look at what types of data exfiltration that have been postulated in theory but yet to be applied. Attempt to come up with a viable algorithm, this will probably lead to a irritative methodology when creating and implementing them as each iteration will improve on follies of the last.

7. Facilities and resources

The only other resources that would be useful to have is dummy personal or sensitive data to use when exfiltrating the data so that the program can be put through its paces with large amounts of data similar to what would be exfiltrated in the real world.

8. Log of risks

Risk description and type	Risk impact - description	Risk probability (severity x likelihood)	Mitigation / control	First indicator – that the risk is turning into an issue
Source code loss	Having to rewrite all the code written up to this point	High severity Low likelihood	Keep 2 backups of the code 1 on site and 1 in the cloud.	If one of the backups fails
Documentation loss	Having to rewrite all of the supporting documentation for the project	High severity Low likelihood	Keep 2 backups of the Documentation 1 on site and 1 in the cloud.	If one of the backups fails
Illness	Not being able to work to full capacity or full excellency	High Severity Low Likelihood	Finish work with large margins before handing dates to allow for recovery of illness.	I start to feel under the weather.

9. Starting point for research.

Starting research topics for this project is; current regularly used method of data exfiltration, any new methods of data exfiltration, the methods of mitigation of data exfiltration, find any dummy data that can be used for exfiltration testing such as personal information e.g. names addresses, phone, email, or credit card information.

10. Breakdown of tasks

1. Write the literature review, this will hopefully inform me on new methods and currently used methods of data exfiltration.

2. Find / make data to exfiltrate could have different types data sets e.g. Credit card info, addresses phone, password data base(worst case)...

3. Develop the software GUI
a. Design the GUI
b. Code the GUI

4. Implement the current exfiltration methods in the software
a. Find the Algorithm for the selected method of data exfiltration
b. Implement the Algorithm to the program

5. Implement new Exfiltration methods
a. Create the methods
b. Write the algorithms to exfiltrated the data.
c. Implement the Algorithms to program.

6. Implement Speed controls to ether exfiltrate the data at top speed or a set speed to evade suspicious.

7. Develop the data reassemble
a. For each method of data exfiltration if the data is mangled beyond human readability by having it in data logs or other forms of data.
b. This will be a part of the GUI and the data exfiltration program.
c. Make sure it works correctly with different data types
d. Build in data redundancy (The 2 generals problem)

8. Test the System
a. Test the data exfiltration works correctly
b. Test that the Data reassemble works correctly.

9. Write up the Documentation
a. Include ability to add more types of data exfiltration

10. Write up the Supporting documents for the dissertation.

11. Project plan

Action	Start date	Completion date
Write the literature review	12/10/19	21/10/19
Find / make data to exfiltrate	21/10/19	28/10/19
Develop the software GUI	28/10/19	02/12/20
Implement current exfiltration methods	02/12/20	20/01/20
Implement new Exfiltration methods	20/01/20	10/02/20
Develop the data reassemble	10/02/20	24/02/20
Implement Speed controls	24/02/20	02/03/20

Test the System	02/03/20	12/03/20
Write up the Documentation	13/03/20	27/03/20
Write up documents for the dissertation.	14/10/19	27/03/20

12. Legal, ethical, professional, social issues (mandatory)

The major ethical problem that could arise is that as I am developing different method of exfiltrating data that has not been used. If someone got there hand on the program that they then used to steal data from an organisation. But if I was to publicly publish any new methods found then the information could be used to come up with countermeasure for the data exfiltration.